

CoRE Stack Backend — Pipeline Verification & Execution Tracing Manual

This manual serves as a comprehensive, execution-oriented, and beginner-friendly debugging handbook for senior developers and backend engineers onboarding onto the CoRE Stack repository. It details how to verify pipelines end-to-end, trace data flows through the distributed architecture, validate intermediate and final outputs, and troubleshoot runtime failures.

PART 1 — Understanding Pipeline Verification

In standard web applications, verification involves testing if a REST API writes a record to a database and returns a 200 OK JSON response. In **geospatial backend systems**, this simple definition is insufficient.

1.1 What is "Pipeline Verification"?

Pipeline verification is the rigorous process of confirming that:

- 1. **The Orchestration Layer** successfully registers a request and offloads it asynchronously.
- 2. **The Compute Layer** successfully initializes, connects to the data provider, reads spatial region assets, and processes raster/vector operations.
- 3. **The Data Storage Layer** successfully exports the computed layers to persistent cloud or database assets.
- 4. **The Map Server Layer** imports, registers, and styles the generated layers for high-performance map tiling.
- 5. **The Metadata Registry** indexes all records, ensuring they are spatially and temporally queryable by downstream services.

1.2 Why Verification Matters in Geospatial Systems

Geospatial computations are exceptionally expensive, scale to gigabytes per query, and heavily rely on external APIs (Google Earth Engine, GeoServer, AWS S3). A failure in one micro-segment of the pipeline often results in **silent failures**:

- The API returns a 200 Success code, but the generated shapefile is empty.
- The GEE task succeeds, but GeoServer fails to download the resulting GeoTIFF, leaving a blank tile on the user's dashboard.
- The DB records are written, but the sync_to_geoserver flag is False, hiding the layer from the mobile application.

1.3 What "Correct Execution" Means Architecturally

Architecturally, correct execution means that every component completes its lifecycle transitions successfully, matching the sequence below:

[Client Request] —> [Django DB Registry] —> [Celery Queue] —> [GEE Compute Engine]

|

[Dashboard WMS] <— [GeoServer Layer] <— [PostGIS Metadata] <— [GEE Export Asset]

1.4 Failure Modes in CoRE Stack

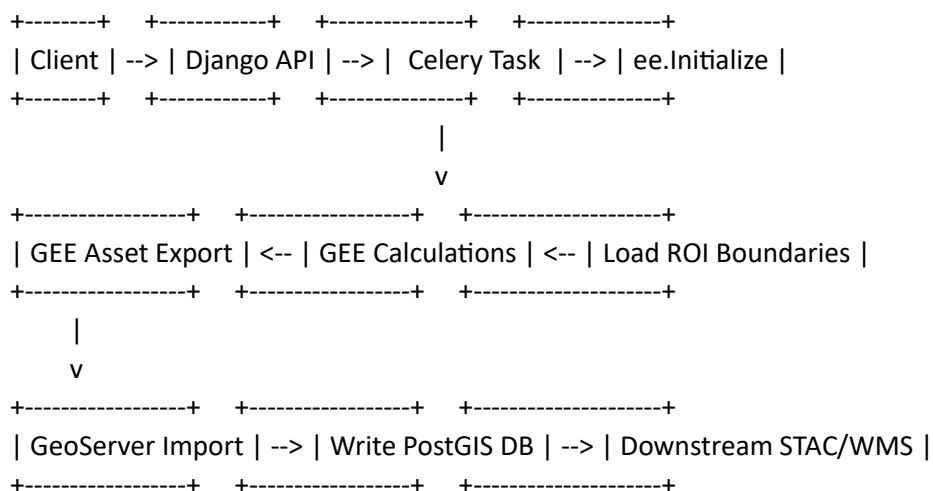
Here are the primary failure categories and their symptoms:

Failure Mode	Primary Cause	Symptom	How to Identify
API Failure	Invalid body parameters, unauthenticated tokens, missing DB boundaries	400 Bad Request, 405 Method Not Allowed, or 500 Server Error	Django log traceback / stdout

Failure Mode	Primary Cause	Symptom	How to Identify
Celery Failure	Worker offline, incorrect queue subscription, solo pool block	Tasks remain in PENDING state, no worker pickup	Celery inspect active or Redis queue size
GEE Init Failure	Invalid/expired Fernet key, missing service account JSON	Earth Engine client library not initialized error	Celery worker console log
GEE Export Failure	GEE export queue limits, out of memory, timeout	GEE task state changes to FAILED or CANCELLED	ee.data.listOperations() or GEE Console
GeoServer Sync Failure	Invalid SLD styles, shapefile generation crashes, REST auth error	GEE task succeeds but layer does not show on WMS	GeoServer log /geoserver/logs/ or 500 REST API response
Metadata Mismatch	Database primary key mismatch, block ID not matching SOI ID	Layer is published but GET /api/get_gee_layer/ returns empty	PostgreSQL query on computing_layer
Empty Outputs	No cloud-free satellite imagery in temporal range, incorrect clipping	Published vector layer has 0 features or rasters are entirely black	QGIS layer preview / GEE inspector

PART 2 — Generic End-to-End Pipeline Flow

Every geospatial computation pipeline in CoRE Stack follows a structured, modular design pattern. Understanding this sequence is vital to pinpointing execution bottlenecks.



2.1 API Ingestion (Urls & Views)

- **Purpose:** Capture the request parameters (state, district, block, start_year, end_year, gee_account_id) and validate authentication.

- **Involved Files:** `computing/urls.py` and `computing/api.py`.
- **Expected Logs:** `POST /api/v1/generate_ci_layer/ HTTP/1.1 200 OK`
- **Common Failures:** Parameter validation errors (e.g. state SOI ID not found in database).

2.2 Celery Enqueueing

- **Purpose:** Instantly offload the blocking GEE task to Redis so the HTTP thread remains free.
- **Involved Files:** `nrm_app/celery.py` and `computing/cropping_intensity/cropping_intensity.py` (or equivalent).
- **Expected Behavior:** The view invokes the task with `.delay()` or `.apply_async()`, returning a Celery task UUID in the response body.
- **Verification Method:** Check the Redis queue using Python redis package: `r.llen("celery")`.

2.3 ROI (Region of Interest) Spatial Loading

- **Purpose:** Load the administrative boundary shapefile (TehsilSOI/DistrictSOI/MWS) into Earth Engine to restrict calculations.
- **Involved Files:** `utilities/gee_utils.py` and `utilities/constants.py`.
- **Expected Behavior:** Resolves administrative names into GEE Asset directories (e.g., `projects/core-stack-org/assets/mws/rajasthan/...`).
- **Common Failures:** `ee.ee_exception.EEException: Asset '...' not found`.

2.4 Earth Engine Computation

- **Purpose:** Perform high-throughput satellite analysis (NDVI, LULC masking, NDWI indexing) in the Google Cloud.
- **Expected Behavior:** Translates Python GEE client calls into standard Google GEE JSON API requests. No heavy raster processing occurs on the local CPU.

2.5 Exporting to GEE Asset

- **Purpose:** Write GEE memory computations to a persistent GEE FeatureCollection or Image asset.
- **Expected Logs:** Export task submitted to GEE. Task ID: `OQJ456N2L3X7...`
- **Verification Method:** Run `ee.data.listOperations()` to monitor progress.

2.6 GeoServer Syncing & WMS Layer Registration

- **Purpose:** Expose the resulting dataset as an OGC WMS/WFS map layer.
- **Involved Files:** `utilities/geoserver_utils.py` and `computing/urls.py`.
- **Expected Behavior:** Converts the GEE asset to a shapefile or GeoPackage, uploads it to GeoServer via REST, and applies custom SLD styles.
- **Common Failures:** Styling errors or REST credentials unauthorized.

2.7 Database Metadata Save & Downstream Indexing

- **Purpose:** Write the final metadata to PostgreSQL so API clients can instantly query the layer.
- **Involved Files:** `computing/models.py` (Layer & Dataset tables).
- **Expected Behavior:** A new row is appended with the absolute GeoServer WMS URL, GEE asset ID, temporal bounds, and `sync_to_geoserver=True`.

PART 3 — Where to Start Reading Code

Tracing an unknown geospatial pipeline can feel overwhelming. Follow this step-by-step reading methodology to dissect any CoRE Stack pipeline from scratch.

3.1 Step 1: Locating the Entry Route

Open `nrm_app/urls.py` and find the main API router:

```
python
path("api/v1/", include("computing.urls")),
```

This tells you that all geospatial computation entry points live within the computing/ application directory.

3.2 Step 2: Mapping URLs to Views

Open computing/urls.py and search for the pipeline path. For example, for the Drought pipeline:

python

```
path("drought_index/", api.generate_drought_layer, name="drought_index"),
```

This routes HTTP traffic directly to the generate_drought_layer view function inside computing/api.py.

3.3 Step 3: Deconstructing the View

Open computing/api.py and search for the view function. Highlight the following components:

- **Parameter Reading:** How does it parse incoming JSON data?
- **Celery Offloading:** Look for the task invocation. In the Drought pipeline:

python

```
task = generate_drought.delay(state=state, district=district, block=block, ...)
```

- **HTTP Response:** Notice that the view immediately returns a 200 Success response with the task.id without waiting for GEE or Celery to finish.

3.4 Step 4: Finding the Celery Task

All pipelines have separate folders

inside computing/ (e.g. drought/, cropping_intensity/, terrain_descriptor/).

Open computing/drought/drought.py (or similar) and search for the @app.task decorator:

python

```
@app.task(bind=True)
```

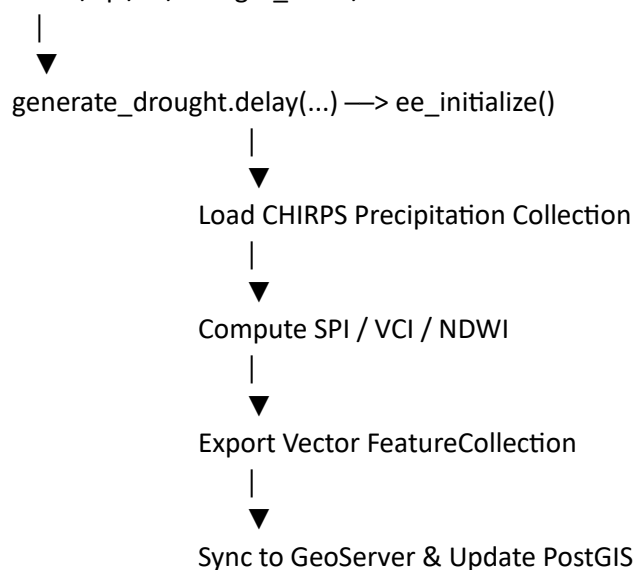
```
def generate_drought(self, state, district, block, ...):
```

This is the **true starting point of background execution**. Dissect the file by scanning only the functions triggered sequentially inside this task block.

PART 4 — Drought Pipeline Deep Verification

The Drought pipeline measures regional meteorological drought indexes by cross-analyzing CHIRPS daily precipitation data against historical baselines.

POST /api/v1/drought_index/



4.1 Step-by-Step Code Walkthrough

Stage 1: API Endpoint Ingestion

- **Endpoint:** POST /api/v1/drought_index/

- **View Function:** generate_drought_layer() inside computing/api.py.
- **Required Parameters:**

```
json
{
  "state": "Himachal Pradesh",
  "district": "Bilaspur",
  "block": "Bilaspur",
  "year": 2023,
  "gee_account_id": 1
}
```

Stage 2: Background Task Initialization

The Celery task generate_drought() starts inside computing/drought/drought.py:

```
python
ee_initialize(gee_account_id)
# Resolves administrative names to GEE vector assets
roi_path = get_gee_dir_path([state, district, block], GEE_PATHS["MWS"]["GEE_ASSET_PATH"])
roi = ee.FeatureCollection(roi_path)
```

Stage 3: Data Aggregation & Calculations

The pipeline queries Earth Engine to calculate drought indicators per micro-watershed:

1. **SPI (Standardized Precipitation Index):** Evaluates rainfall deficits using the **CHIRPS Daily precipitation collection** (UCSB-CHG/CHIRPS/DAILY).
2. **VCI (Vegetation Condition Index):** Compares current crop vigor against historical NDVI minimums and maximums using Sentinel-2 or MODIS imagery.
3. **Soil Moisture Indices:** Leverages Sentinel-2 NDWI imagery to evaluate plant water stress.

Stage 4: Export & Sync Lifecycle

```
python
task_id = export_vector_asset_to_gee(roi_drought, description, asset_id)
check_task_status([task_id]) # Recursive polling loop (sleep 60s)
make_asset_public(asset_id)
save_to_db_and_sync_to_geoserver(layer_name, asset_id, year, state, district, block)
```

4.2 Verification & Debugging Signals

- **Healthy Log Pattern:**

```
[INFO] Celery task received: generate_drought
[INFO] ee initialized successfully.
[INFO] Drought calculations computed. Task ID submitted to GEE: ABCD1234
[INFO] Polling GEE task ABCD1234... Status: RUNNING
[INFO] Polling GEE task ABCD1234... Status: SUCCEEDED
[INFO] Syncing shapefile to GeoServer workspace 'nrm'...
[INFO] Database metadata saved successfully. Layer ID: 45
```

- **Broken Log Pattern (rainfall data missing / temporal error):**

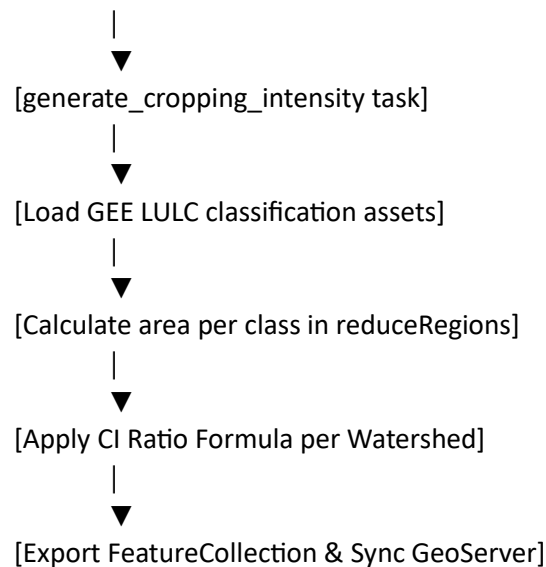
```
[ERROR] ee.ee_exception.EEException: Collection.reduce: No bands available for reduction in CHIRPS
rainfall query.
```

PART 5 — Cropping Intensity Pipeline Deep Verification

The Cropping Intensity pipeline evaluates seasonal agricultural patterns (Kharif, Rabi, Zaid) to determine overall cropping intensity per micro-watershed.

5.1 Step-by-Step Code Walkthrough

[POST /api/v1/generate_ci_layer/]



Stage 1: API Endpoint Ingestion

- **Endpoint:** POST /api/v1/generate_ci_layer/
- **View Function:** generate_ci_layer() in computing/api.py.
- **Required Parameters:**

json

```
{
  "state": "Himachal Pradesh",
  "district": "Bilaspur",
  "block": "Bilaspur",
  "start_year": 2021,
  "end_year": 2022,
  "gee_account_id": 1
}
```

Stage 2: Fetching seasonal LULC Layers

Open computing/cropping_intensity/cropping_intensity.py and trace the GEE LULC map loading sequence:

python

```
while s_year <= end_year:
```

```
    # Loads classified satellite rasters representing different cropping seasons
```

```
    lulc_js_list.append(ee.Image(get_gee_dir_path(...) + f"{asset_suffix}_{s_year}_LULCmap_10m"))
```

Stage 3: Spatial Index Reduction (reduceRegions)

The pipeline runs pixel-area calculations within each micro-watershed boundary:

python

```
roi = forestArea.reduceRegions(
    collection=roi,
    reducer=ee.Reducer.sum(),
    scale=10,
    tileScale=4
)
```

This reduces massive 10m rasters into small, tabular properties (attributes) appended to each micro-watershed vector feature.

Stage 4: Index Calculation

Tabular values are used in a mapping function to compute the Cropping Intensity Index:

```
python
cropping_intensity =
single_cropped.add(double_cropped.multiply(2)).add(triple_cropped.multiply(3)).divide(total_croppable_area)
```

PART 6 — Terrain Descriptor Pipeline Verification

The Terrain Descriptor pipeline generates topographic baselines (elevation, slope, and aspect) utilizing the Shuttle Radar Topography Mission (SRTM) DEM.

6.1 Flow & Code Architecture

[SRTM 30m Digital Elevation Model] → [ROI Clipping] → [Slope & Aspect calculations]

[GeoServer Raster WMS] ← [Upload raw GeoTIFF] ← [GCS Bucket] ← [Export Image to GCS]

- **Endpoint:** POST /api/v1/generate_mws_layer/
- **File Location:** computing/terrain_descriptor/terrain_descriptor.py

Unlike Drought and Cropping Intensity which export tabular vector shapefiles, the Terrain pipeline handles **massive raster grids (GeoTIFFs)**:

1. **DEM Extraction:** Loads USGS/SRTMGL1_003 (30m elevation model).
2. **Topographic Slopes:** Uses Earth Engine algorithms:

```
python
```

```
slope = ee.Terrain.slope(srtm)
```

```
aspect = ee.Terrain.aspect(srtm)
```

3. **Image Export to GCS:** Because DEM raster datasets are extremely large, the pipeline exports them to Google Cloud Storage (GCS) as GeoTIFF files:

```
python
```

```
task = ee.batch.Export.image.toCloudStorage(
```

```
    image=dem_composite,
    bucket=GCS_BUCKET_NAME,
    description=description,
    scale=30,
    region=roi.geometry()
```

```
)
```

4. **GeoServer Sync:** Downloads the GeoTIFF from the GCS bucket to the local server, and publishes it using GeoServer's raster coverage REST APIs.

PART 7 — Surface Water Bodies (SWB) Pipeline Verification

The SWB pipeline traces surface water dynamics over multiple seasons to map permanent and seasonal water bodies.

7.1 Pipeline Mechanics & Detection Logic

- **Endpoint:** POST /api/v1/generate_swb/
- **File Location:** computing/surface_water_bodies/surface_water_bodies.py

Water Indexing Calculations

The pipeline cross-references **JRC Global Surface Water** records and monthly Sentinel-2 composites. It identifies water pixels using the **Modified Normalized Difference Water Index (MNDWI)**: Seasonal variations are calculated across two main water body layers:

- **Permanent Waterbodies:** Pixels classified as water in all multi-season satellite acquisitions.

- **Seasonal Waterbodies:** Pixels containing water only during monsoon or post-monsoon months.

Verification Indicators

- **Successful Execution:** The generated vector map contains custom polygons for lakes, reservoirs, and rivers, styled with a distinct blue scheme on GeoServer.
- **Broken Execution (Zero-Output Failure):** The pipeline outputs an empty shapefile due to a lack of cloud-free imagery during winter months.

PART 8 — How to Verify Every Layer of the System

This layer-wise checklist provides a systematic method to locate failures across the stack.

8.1 A. Django / API Layer Verification

1. **Endpoint Access:** Run a quick login request to confirm the API is active:

bash

```
curl -X POST http://localhost:8000/api/v1/auth/login/ -H "Content-Type: application/json" -d '{"username\":\"admin\",\"password\":\"adminpass\"}'
```

2. **Logs Monitoring:** Open the Django console. Ensure incoming requests route successfully (200 or 201 HTTP statuses).
3. **Common Failures:** 400 Bad Request implies missing or misspelled parameters. 403 Forbidden indicates the JWT token has expired or is missing from the header.

8.2 B. Celery Layer Verification

1. **Worker Status:** Verify if Celery is running and listening:

bash

```
celery -A nrm_app status
```

2. **Queue Monitoring:** Use Redis CLI to check the length of queued tasks:

bash

```
redis-cli llen celery
```

3. **Worker Log Tracing:** Keep the Celery worker window open. Look for:
 - Received task: computing.drought.tasks.generate_drought[...]
 - Task succeeded in X.XXs: layer_at_geoserver=True

8.3 C. GEE Layer Verification

1. **Check Real-time GEE Tasks:** Run a Python snippet to print GEE tasks directly:

python

```
import ee
```

```
ee.Initialize()
```

```
print(ee.data.listOperations()[3])
```

2. **Visual Asset Check:** Open the Google Earth Engine Code Editor. Search for the generated asset in the "Assets" tab (e.g. users/corestack/rajasthan/drought_2023). Ensure permissions allow public read access.

8.4 D. GeoServer Layer Verification

1. **REST Admin Portal:** Navigate to <http://localhost:8080/geoserver> in a browser. Log in with admin credentials.
2. **Preview Layers:** Go to "Layer Preview". Search for the target workspace (nrm) and click "OpenLayers" next to your generated layer. If it displays on the map, GeoServer synchronization was completely successful!
3. **Workspace Check:** Ensure the default workspace nrm exists in the administrative dashboard.

8.5 E. PostgreSQL / PostGIS Database Verification

1. **Connect to Database:** Log into the database (e.g. using psql or pgAdmin).
2. **Query Layers Table:**

```
sql
```

```
SELECT id, name, layer_name, sync_to_geoserver, is_stac_specs_generated FROM computing_layer  
ORDER BY id DESC LIMIT 5;
```

3. **Confirm Geoadmin Boundaries:** Ensure that district/block SOI geometry indices are not empty:

```
sql
```

```
SELECT count(*) FROM geoadmin_tehsilsoi;
```

PART 9 — Expected Outputs & Correctness Signals

It is vital to distinguish between a technically successful database insert and a scientifically correct geospatial result.

9.1 Naming Conventions and Locations

Pipeline	Output Format	Output Directory / GEE Asset Path	Naming Pattern
Drought	Vector Shapefile	projects/core-stack/assets/mws/{state}/{dist}/{block}/drought_{year}	{district}_{block}_drought_{year}
Cropping Intensity	Vector Shapefile	projects/core-stack/assets/mws/{state}/{dist}/{block}/cropping_intensity_{year}	{district}_{block}_intensity
Terrain Descriptor	Raster GeoTIF	geoserver/workspaces/nrm/coverages/srtm_{block}/	srtm_{district}_{block}
Surface Water	Vector Shapefile	projects/core-stack/assets/mws/{state}/{dist}/{block}/swb_{year}	{district}_{block}_swb

9.2 What Healthy Outputs Look Like

- **Tabular Attributes:** Every micro-watershed polygon should have calculated values (e.g., cropping_intensity_2022 containing numbers between 0.5 and 2.5).
- **Spatial Alignment:** In QGIS or the dashboard map, layers must snap exactly onto the official Survey of India (SOI) administrative block lines.
- **Metadata Alignment:** The database computing_layer entry has a valid UUID, links to a correct dataset, and has sync_to_geoserver = True.

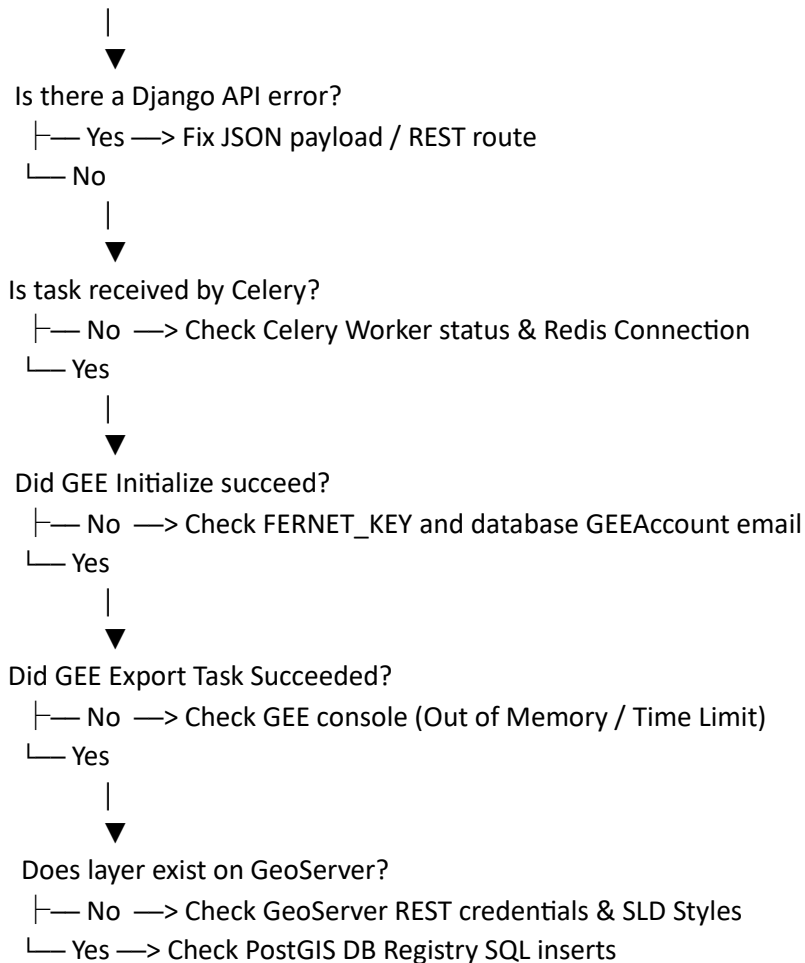
9.3 Suspicious Signals (Silent Failures)

- **All-Zero Rasters:** Terrain outputs having a constant pixel value of 0. This implies the elevation DEM did not clip correctly or could not read SRTM datasets.
- **Empty Vectors:** A GeoPackage shapefile with zero rows. This indicates the pipeline completed its cycle but the satellite image queries returned only null observations (commonly due to excessive cloud cover).
- **Missing Metadata:** The layer exists on GeoServer, but there is no registering database entry. Downstream services will never know the layer exists.

PART 10 — Runtime Debugging Guide

When a pipeline fails, follow these diagnostic loops to isolate and fix the root cause.

[Pipeline Failure]



10.1 Diagnostic Runbooks

Celery Task Stuck / Pending

- **Cause:** Celery is waiting for a slot, or the worker crashed.
- **Runbook:**
 1. Inspect worker processes: `celery -A nrm_app inspect active`.
 2. If on Windows, ensure the `-P solo pool` flag was passed. Django Celery on Windows cannot run multiprocessing child nodes without it.

GEE Cryptography Fernet Error

- **Cause:** `FERNET_KEY` is not a 32-byte URL-safe base64 string, or Django is unable to read the environment variables.
- **Runbook:**
 1. Generate a valid key: `from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())`.
 2. Update the key inside `nrm_app/.env` and restart Django and Celery.

GeoServer WMS Layer Blank (404 / 500 REST calls)

- **Cause:** The workspace `nrm` is missing or the uploaded GeoTIFF format is unsupported.
- **Runbook:**
 1. Open GeoServer Admin Panel (<http://localhost:8080/geoserver>).
 2. Under "Workspaces", ensure `nrm` is added.
 3. Verify that the REST username/password inside `nrm_app/.env` matches GeoServer's global security credentials.

A standard CoRE Stack pipeline module contains hundreds of lines of complex Earth Engine logic. Here is how to digest these files without getting overwhelmed.

11.1 The Anatomy of a Pipeline File

Every single pipeline file is divided into distinct, predictable execution blocks:

```
+-----+
| SECTION 1: imports & Celery Decorators          |
| - imports ee, nrm_app.celery, constants, utils  |
| - @app.task(bind=True) definition              |
+-----+
| SECTION 2: Initialization & ROI Clipping        |
| - ee_initialize(gee_account_id)                 |
| - Load administrative boundaries using geoadmin model attributes |
+-----+
| SECTION 3: Image / Feature Calculations          |
| - Loads satellite imagery collections (Sentinel, Landsat) |
| - Maps algorithms (indices like NDVI, SPI, NDWI) over lists |
+-----+
| SECTION 4: persistent Export                    |
| - Submits ee.batch.Export tasks back to Google Cloud |
| - Blocks execution with check_task_status() polling loop |
+-----+
| SECTION 5: GeoServer Registration & Database Registry Updates |
| - Downloads shapefile/GeoTIFF, sends to GeoServer, registers in DB |
+-----+
```

11.2 What You Can Initially Ignore

When reading a pipeline code file for the first time:

- **Ignore GEE reduction mathematics:** Do not spend hours dissecting GEE functions like `ee.Image.pixelArea().updateMask(...)`. Treat them as a black box that yields a classified raster or vector map.
- **Ignore local variable builders:** Ignore GEE path normalization routines. They simply convert names to GEE directory strings.
- **Focus on the boundaries:** Focus strictly on the entry parameters, the export mechanism, and the save database logic.

PART 12 — Reusable Operational Checklists

Use these checklists for daily developer operations and production deployments.

12.1 Services Startup Checklist

- **Docker Engine:** Docker Desktop or Docker daemon is running.
- **Database & Redis Containers:** Spin up local containers:

bash

`docker-compose up -d`

- **Conda Environment:** Activate the corestack conda workspace:

bash

`conda activate corestack-backend`

- **Django Server:** Run in a terminal workspace:

bash

`python manage.py runserver 0.0.0.0:8000`

- **Celery Worker:** Run Celery worker:

- *Windows:* celery -A nrm_app worker -l info -P solo
- *Linux/MacOS:* celery -A nrm_app worker -l info --concurrency=4

12.2 Pipeline Run Checklist

- Confirm Django /status/ dashboard displays all green checkmarks.
- Check if GEEAccount has credentials successfully seeded in the database.
- Verify administrative boundary SOI maps are seeded.
- Trigger the pipeline via a POST API request.
- Monitor Celery output to confirm GEE task submission.
- Monitor GEE task status until state changes to SUCCEEDED.
- Verify WMS layers are fully visible on the map interface.

PART 13 — Beginner-Friendly Mental Models

Geospatial backend systems can seem complex because they distribute tasks across multiple platforms. Keep these simple analogies in mind to guide your development.

13.1 Mental Model 1: The Restaurant Analogy (Django & Celery)

- **The Client (Customer):** Sits at the table and orders a complicated dish that takes 45 minutes to cook (e.g. a cropping intensity pipeline).
- **Django (The Waiter):** Takes the order, writes a ticket, passes it to the kitchen, and tells the customer: *"Got your order! Here is your receipt number (Task ID). I'll let you know when it is ready."* The waiter is instantly free to serve other tables.
- **Celery (The Kitchen Chef):** Picks up the ticket from the counter (Redis Broker) and cooks the dish in the background. If the chef is busy, the ticket waits in line (Queue).

13.2 Mental Model 2: The Projector Analogy (GEE & GeoServer)

- **Google Earth Engine (The Film Roll):** GEE stores petabytes of imagery and handles the processing. It is like the film roll containing all frames. However, GEE is not built to serve direct web traffic to thousands of browser clients simultaneously.
- **GeoServer (The Film Projector):** Django extracts the calculated film from GEE, prints it to a standardized slide (Shapefile/GeoTIFF), and loads it into the GeoServer projector. GeoServer projects the maps directly onto the user's dashboard screen (WMS Tiles) quickly and efficiently.
- **PostgreSQL (The Movie Catalog):** Indexes all movie slides so the projector knows which slide belongs to which block and year.

This handbook provides the fundamental tools to trace, verify, and debug any pipeline in the CoRE Stack codebase. Relate every runtime error back to these core layers, follow the sequencing workflows, and use the diagnostic checklists to systematically resolve system issues.